Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

09/09/2022

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [ ] Zanero [ ] Carminati [ ] Barenghi

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Priority Grading [ ] Yes, Motivation:

[ ] No

Instructions

- **Duration**: 2.5 hours.
- The exam is composed of 17 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct exam delivery.
  - If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
  - We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
  - Regarding the submission:
    - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
    - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
    - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
    - We will evaluate only the uploaded **readable** documents.
    - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider modern active biomedical implantable devices (i.e., embedded biomedical devices placed in the human body during surgery or other clinical intervention to serve a specific function relying for their functioning on a source of power), such as the “artificial cardiac pacemaker”.
The patient can monitor (i.e., read-only mode) the data transmitted from the devices (e.g., battery and device status, heart rate, medical records) with a mobile app that sends data to a cloud back-end, which stores and collects data.
Such devices can be recharged through a specific wireless charging device available to the patient.
In addition, these implantable devices must communicate with stations located in hospitals and accessible only by authorized personnel to allow the update of the firmware of the device or for tuning their functionalities.
The communication between the endpoints and the implantable device is guaranteed through a low-energy Bluetooth connection encrypted with symmetric encryption (the key is pre-shared and hardcoded during the device's production). The mobile application has different keys with respect to the hospital stations.

[1 point] 1. What are the main assets at risk in such a scenario? Suggest at least two assets.

Asset	**Motivation**
The life of the patient	**[...]**
Patient’s personal data or reputation	**[...]**

[1 point] 2. Suggest, in rough order of importance, the most likely potential threats and threat agents against such infrastructure and their operating companies.

Threats	**Threat Agents**
- **DOS** **Motivation:** **[...]**	- **Targeted attacker** **Motivation:** **[...]**

- **Spoofing/sniffing** **Motivation:** **[...]**	- **Competitor** **Motivation:** **[...]**

[2 points] 3. A company working in the field is proposing to substitute the symmetric encryption currently deployed with asymmetric encryption. What are the advantages and disadvantages of the proposed solution specifically considering the scenario under analysis?

Advantages	**Disadvantages**
- **Key management (see slides)**	- **Computational complexity** - **Power consumption** - **PKI limitations**

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] 1. Consider the following authentication system. Each legitimate user generates a keypair for an asymmetric *encryption* scheme, and uploads on the server which should be authenticating her the corresponding public key. To get authenticated, the user draws a random string *r*, decrypts it with her own *private* key obtaining a string *s*, and sends the pair *(r,s)* to the server over a confidential and integrity preserving channel. The server encrypts *s* with the user's public key and checks if the result matches *r,* in which case it authenticates the user.
Argue on whether this system is providing proper authentication, either justifying why it is secure, or showing an attack and proposing a working countermeasure. To this end, consider an attacker which comes into possession of a user's public key k_pub.

Solution: The authentication system can be broken in the following way. The attacker picks a random string *s'*, encrypts it with k_pub obtaining *r'*, and sends *(r',s')* to the server. Switching the asymmetric primitive from an encryption to a signature scheme fixes the problem, as an attacker would need to randomly guess a valid signature string which can be verified with the user's public key.

[3 points] 2. Consider the following password-based authentication mechanism: you mandate that the user inputs 6 words, uniformly randomly drawn from an English dictionary (containing 2^14 words). Whilst your users have been trained to randomly pick the words, you want to put up an extra layer of defenses, locking an account after some number *n* of failed attempts.
Consider the following scenario: one user every eight picks two words from the dictionary at random, and repeats them 3 times.
What is the value of *n* capping the probability of a successful sequence of *n* guesses to at most 1 in a billion? Justify your answer.
Describe a simple check upon password enrollment/renewal, which is more effective than the guessing cap, and quantify its effectiveness.

Solution: Notice that the system does not prevent an attacker from trying to guess the passwords of multiple accounts simultaneously: the accounts will be locked only when the failed attempts against a single account are more than n. As a consequence, we can assume that the attacker will always be hitting at least an account of a user with poor password policies. This results in a single guess succeeding with probability (2^(-14))^2 = 2^-28, which is already higher than our target (~2^-30). No cap can improve this. A simple check upon password enrollment is to test that at least a subset of the words composing the password are different. While this reduces the possible passwords, it does so by a negligible amount.
As an example consider testing that at least 4 words are different: this reduces the number of potential passwords from 2^84 to 2^84-(2^14+20*2^28+30*2^42) < 2^84-(32*2^42) = 2^84-(2^47), which is still way larger than 2^83, thus definitely large enough.

Exercise 3 - Binary Vulnerabilities (6 points)

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04 #include <time.h>
05
06 typedef struct {
07   char buf1[4];
08   char buf2[4];
09   time_t t;
10 } data_t;
11
12
13 void gen_random(char *s, const int len) {
14     static const char alphanum[]=
15 "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
16     for (int i = 0; i < len; ++i) {
17         s[i] = alphanum[rand() % (sizeof(alphanum) - 1)];}
18
19 }
20
21 void random_stuff(data_t * data) {
22     char buf3[64];
23
24     scanf("%63s", buf3);
25
26     printf("%s", buf3);
27     if (strncmp(buf3, "*Message*", 9) == 0)
28         printf(buf3);
29     else
30         abort();
31 }
32
33 void main(int argc, char** argv) {
34     data_t data;
35
36     srand((unsigned) time(&data.t));  // Initializes random number generator
37     gen_random(data.buf2, 4); // Generate a random string data.buf2
38     scanf("%8s", data.buf1);
39     random_stuff(&data);
40
41     if (strncmp(data.buf1, data.buf2, 4) == 0)
42         system("/bin/sh\0");
43 }
```

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.

- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual `cdecl` calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against exploiting memory corruption errors are present (unless specified otherwise).*

From now on, assume that the "Stack Canary" mitigation technique is correctly implemented (i.e., it can not be guessed or copied)

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Motivation**
Buffer Overflow	38	[See slides]
Format String	28	[See slides]

[3 points] 2. Considering the detected vulnerabilities, write an exploit that **must execute the following shellcode**, composed of 16 bytes, which opens a shell: `0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90 0x10 0x20 0x30 0x40 0x50 0x60 0x70 0x80`

2.a) Which of the vulnerabilities (or combination) would you exploit**?**

It is not possible to exploit the buffer overflow vulnerability since the total amount of bytes that can be overflowed are 8. Also, Stack canary is correctly implemented.

The format string can be exploited to overwrite the SEIP with the address of the shellcode placed in buf3 (or in an ENV variable).

2.b) Describe **all the steps and assumptions** required to exploit the vulnerability successfully. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution.

Exploit the format string vulnerability or a debugger to estimate the displacement of variables on the stack: position of buf3, the canary, the sEIP. Also, the memory must be prepared to fill buf3 with the format string exploit and the shellcode or just put the shellcode in an env variable. Then, estimate the position of each "component". Finally, you need also to pass the condition at line 27.
The format string exploit will have the following main components:

- target: address of sEIP
- What to write: address of shellcode
- Displacement and char already written 8+9+3(padding)

Previous lines: [not relevant]
Line 24: Insert *message* + the format string exploit in buf3 followed by shellcode to execute.
Line 28: The format string is exploited and it will overwrite the sEIP with the addr of the location of the shellcode.

2.c) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

Low					
^	buf3[0-3] *mes	<-ESP	random_stuff()	buf3[0-3] *mes	
|	buf3[4-7] sage		random_stuff()	buf3[4-7] sage	
|	buf3[8-11] *AAA		random_stuff()	buf3[8-11] *AAA	
|	buf3[8-11] [FS_exploit]		random_stuff()	buf3[8-11] [FS_exploit]	
|	... [FS_exploit]		random_stuff()	... [FS_exploit]	
|	... [shellcode]		random_stuff()	... [shellcode]	A
|	... [shellcode]		random_stuff()	... [shellcode]	
|	buf3[60-63] [shellcode]		random_stuff()	buf3[60-63] [shellcode]	
|	canary		random_stuff()	canary	
|	sEBP	<-EBP	random_stuff()	sEBP	
|	sEIP		main()	~ A	
|	* data		main()	* data	
|	buf1[0-3]		main()	buf1[0-3]	
|	buf2[0-3] [not relevant]		main()	buf2[0-3] [not relevant]	
|	t		main()	t	
|	canary		main()	canary	
|	sEBP		main()	sEBP	
|	sEIP			sEIP	
|	argc			argc	
|	argv			argv	
High					

[1 point] 3. A friend of yours suggests that there is an alternative way **to exploit the vulnerabilities you found** to spawn a privileged shell in the program **without executing any shellcode or calling library functions**. If it is possible, please explain **all the steps and assumptions** required for successful exploitation. If it is ineffective, please explain why

Exploit the buffer overflow vulnerability at line 38 to overflow buf1 and buf2 to satisfy
`strncmp(data.buf1, data.buf2, 4) == 0`

[1 points] 4. Consider remote exploitation (i.e., you do not have local access to the machine on which the vulnerable program is executed, not SSH, you can interact only with inputs and outputs). Are the previous exploits at point 2. and 3. still working without any modification? If the answer is not, explain why and describe how you would modify the exploit. If the answer is yes, explain, please explain why. Assume that no mitigation is enabled.

Point 2: yes, see slides (unless ENV variable are used)

Point 3: yes, see slides

Exercise 4 - Web Vulnerabilities (6 points)

4TheMeme.com is the best and funniest website for memes. The user can publish new memes or like/dislike memes of other users. Additionally, the website allows the user to upload private memes, i.e., memes that only the owner can see.

The relevant code of the application is the following:

```
00 app.route("/publish_meme") # /publish_meme
01 def publish_meme(request, session):
02     if not is_session_valid(session):
03         return redirect(url_for("/login"))
04     if request.method != "POST":
05         return abort(403)
06     if request.form["CSRF_Token"] != "3DDYMURPHY1MSM4RT":
07         return abort(403)
08     user_id = session["user_id"]
09     img_file = request.form["img_file"]
10     title = request.form["title"]
11     img_path = store_image_disk(img_file, "/images")
12     if request.form["permission"] == "private":
13         is_private = True
14     elif request.form["permission"] == "public":
15         is_private = False
16     else:
17         return abort(403)
18     query = "INSERT INTO MEMES (user_id, title, img_path, is_private) VALUES (‘" +
           user_id + "’, ‘" + title + "’, ‘" + img_path + "’, ‘" + is_private + "’);"
19     db_execute_query(query)
20     return redirect(url_for("/home"))
21
22 app.route("/like_meme") # /like_meme
23 def like_meme(request, session):
24     if not is_session_valid(session):
25         return redirect(url_for("/login"))
26     if request.method != "POST":
27         return abort(403)
28     if request.form["CSRF_Token"] != "3DDYMURPHY1MSM4RT":
29         return abort(403)
30     user_id = session["user_id"]
31     meme_id = request.form["meme_id"]
32     is_liked = check_like_already_done(meme_id, user_id)
33     if is_liked: # Removing the like if already liked
34         remove_like(meme_id, user_id)
35     like_type = request.form["like_type"]
36     if like_type == "like" or like_type == "dislike":
37         query = "INSERT INTO LIKES VALUES (‘%d’, ‘%d’, ‘%s’);"
38         db_execute_query_with_statement(query, meme_id, user_id, like_type)
39     elif like_type != "neutral":
40         return abort(403)
41     return redirect(url_for("/home"))
42
43 app.route("/home") # /home
44 def home(request, session):
45     if not is_session_valid(session):
46         return redirect(url_for("/login"))
47     if request.method != "GET":
48         return abort(403)
```

```
49      query = "SELECT meme_id, title, img_path FROM MEMES WHERE is_private = 'False';"
50      memes = db_execute_query(query);
51      for meme in memes:
52          meme["like"] = get_likes(meme["meme_id"]) # Retrieving likes from DB
53          meme["dislike"] = get_dislikes(meme["meme_id"]) # Retrieving dislike from DB
54      return render_template("home.html", memes=memes)
…
80 # home.html
81 <html>
82      …
83      {% for meme in memes %} # Rendering all the memes in the home page
84          <h1>{{ XSS_Sanitizer(meme["title"]) }}</h1>
85          <img src="{{ XSS_Sanitizer(meme["img_path"]) }}"></img>
86          <p> Likes: {{ meme["like"] }} </p>
87          <p> Dislikes: {{ meme["dislike"] }} </p>
88          <form action="/like_meme" method="POST">
89              <input type="hidden" name="CSRF_Token" value="3DDYMURPHY1MSM4RT"></>
90              <input type="hidden" name="meme_id" value="{{ meme["meme_id"] }}"></>
91              <input type="button" name="like_type" value="like"></>
92              <input type="button" name="like_type" value="dislike"></>
93          </form>
94      {% endfor %}
95      …
96 </html>
```

The relational database schema used by the website is the following:

Table: **USERS**

user_id (Primary Key) (Integer) (Auto increment)	username (String)	email (String)	password_hash (Bytes)

Table: **MEMES**

meme_id (Primary Key) (Integer) (Auto increment)	user_id (Referencing USERS.user_id)	title (String)	img_path (String)	is_private (Boolean)

Table: **LIKES**

meme_id (Primary Key) (Referencing MEMES.meme_id)	user_id (Primary Key) (Referencing USERS.user_id)	like_type (String)

Assume the following:

- The session is correctly handled (i.e., the session cookie is securely generated and managed).
- The function **XSS_sanitizer** correctly escapes all the html entities, i.e., all the html special characters are converted to their equivalent data (> becomes >).
- The **DBMS** does not support stacked queries (e.g., "SELECT … ; INSERT INTO … ;" fails if executed)

- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **store_image_disk** stores the image provided as the first parameter on the disk, in the directory specified as the second parameter. Moreover, the filename of the image is randomly generated with length > 1024 bytes (this prevents other users from accessing private memes). Finally, the function returns the path of the image.

More details:

- The functions **db_execute_query** and **db_execute_query_with_statement** return a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, memes[4]["title"] is accessing the field "title" of the 5th row.
- The function **redirect** redirects the user to the web page of the provided url.
- The function **url_for** generates an url of the provided page of the same domain. Optional query parameters of the url can be specified as parameters.
- The function **render_template** generates an html page with optional parameters that are placed in the html code.
- The function **get_likes** returns the number of likes for the given meme_id. It does execute the query: "COUNT(SELECT * FROM LIKES WHERE like_type = 'like');"
- The function **get_dislikes** returns the number of dislikes for the given meme_id. It does execute the query: "COUNT(SELECT * FROM LIKES WHERE like_type = 'dislike');"
- The function **remove_like** simply removes from the DB a like/dislike made by a user on a meme.

1. [3 points] Based on the available pseudo-code and assumptions, fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding **line/s** of code and relevant data flow/arguments.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
Sql Injection (SQL-I)	*Yes, line 18 in function publish_meme. The input "title" can be used to perform the injection.*	*Use prepared statements.*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*No vulnerabilities for this class*	*No action required*

Cross-site request forgery (CSRF)	*Yes, there's a CSRF Token but it's static. Thus, it is not preventing CSRF from taking place.*	*Use a randomly generated CSRF token.*

2. [1.5 points] One of your friends sent you a link to a strange website (www.evil.com). The page shows you some images and a button that says "Click me". Once clicked, you are redirected to 4TheMeme.com. However, you notice that you like a meme that you previously disliked. Can you reproduce the exploit? Explain all the steps you'd take to reproduce the exploit, including the final code you'd write.

The exploit relies on the CSRF vulnerability due to the static CSRF token. Indeed, the attacker placed a form in the evil.com page where the click of a button performs a POST request to 4TheMeme.com/like page. Such a request will leave a like to a meme and will be executed by the currently logged user, i.e., the victim.

The code of the form is the following (not needed for the solution):

```
<form action="/like_meme" method="POST">
    <input type="hidden" name="CSRF_Token" value="3DDYMURPHY1MSM4RT"></>
    <input type="hidden" name="like_type" value="like"></>
    <input type="hidden" name="meme_id" value="100"></>
    <input type="button" value="Click me"></>
</form>
```

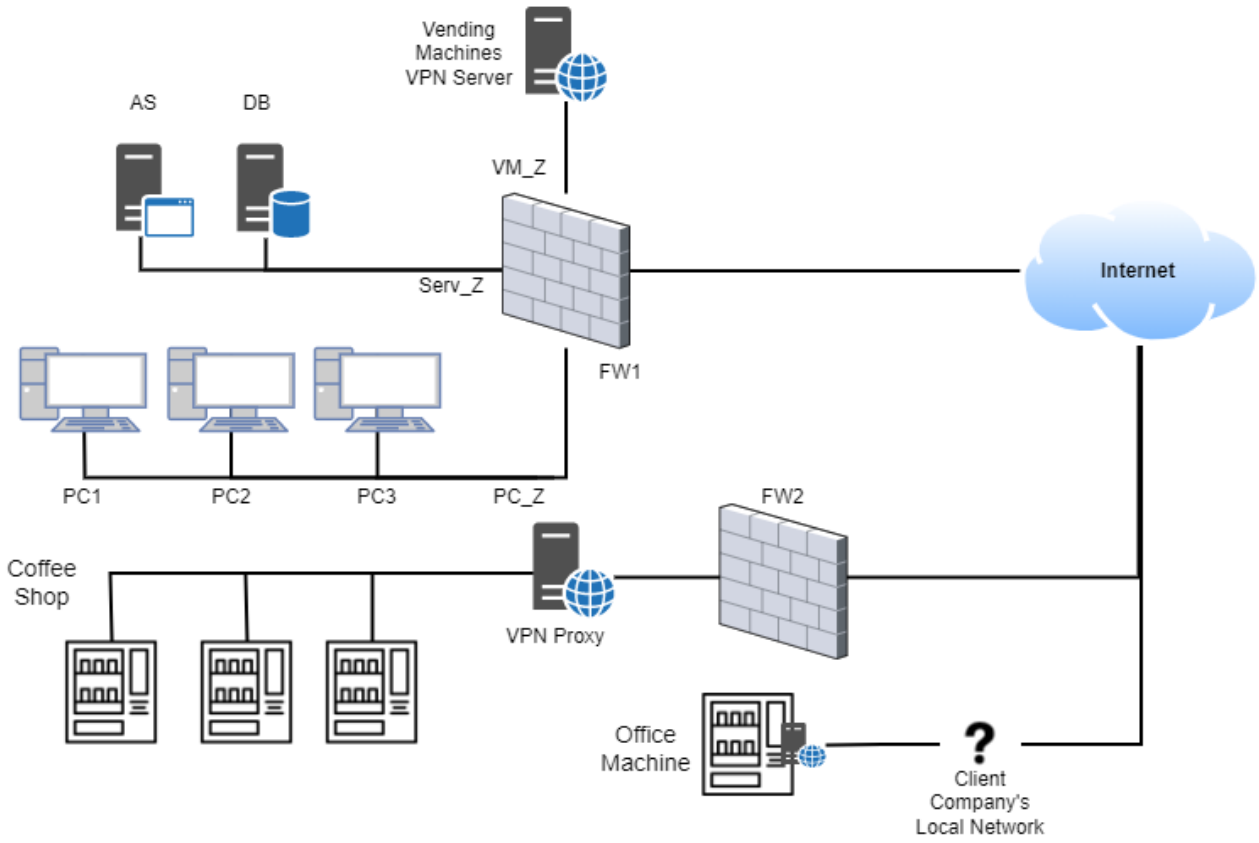
3. [1.5 points] Can you leak a private meme of the user law (user_id = 9). Discuss your exploit in detail and why it works. State all the relevant assumptions and code.

This can be done by exploiting the SQL Injection at line TODO by providing the following meme title:

title = "random_title' , (SELECT img_path FROM MEMES WHERE user_id = 9 AND is_private = 'True' LIMIT 1), is_private = 'False'), (10, 'random_title_2"

Exercise 5 - Network Security (6 points)

The company “SmartSnacks” offers vending machines as a service, both to companies, both as automatic coffee shops with no human personnel. The users can pay with cash, with an NFC card, or directly with their contactless credit cards. In the first case the machine handles the sale in the traditional way. In the second case, the machine reads whether the balance stored in the NFC card is sufficient and, in case, proceeds with the sale and subtracting the credit from the card. In both cases the transaction is handled by the machine and communicated to the backend. In the case of the NFC card, moreover, it is possible to top up its balance by cash through the vending machine (this transaction is stored in the backend too). In the third case the machine communicates with the company's backend to process the payment and obtain authorization for the sale. If the application server confirms the sale to the vending machine, the latter proceeds in providing the chosen product to the user.
The vending machines, when located in an office, are connected to the client's office WiFi through a small embedded computer that connects them to the VPN of the company. The automatic coffee shops comprise usually more than one vending machine, which are all connected through an Ethernet bus to the local network that connects them to the VPN of the company. All communication between the vending machines and the backend uses the HTTP protocol.
The backend, and company's office, is structured as presented in the figure. The employees' computers require complete internet connection, access to the website, which is hosted on AWS, and access to the application server. Moreover, the employees' computers have access to each vending machine for remote debugging and communication. The vending machines require access to the application server to process the payments, which then stores the payment logs in the database. The application server finally needs access to the internet to communicate with the various credit card networks.



1. [2 points] Write the firewall rules, assuming firewalls to be stateful packet filters.

Firewall	Src IP	SRC Port	Direction	Dst IP	Dst PORT	Policy	Description
FW1/FW2	*all*	*any*	*all*	*all*	*all*	*Deny*	*Default Deny*
FW1	*VMx_IP*	*1025-65535*	*VM_Z->Serv_Z*	*AS_IP*	*80*	*Allow*	*Enable communication of payments from vending machines to AS*
FW1	*AS_IP*	*1025-65535*	*Serv_Z->Internet*	*ccn_IP*	*ccn_port*	*Allow*	*Communication between AS and credit card networks (visa/mastercard/...)*
FW1	*PCx_IP*	*any*	*PC_Z->Internet*	*all*	*all*	*Allow*	*PCx Internet communication*
FW1	*PCx_IP*	*1025-65535*	*PC_Z->Serv_Z*	*AS_IP*	*all*	*Allow*	*PCx communication with AS*
FW1	*PCx_IP*	*1025-65535*	*PC_Z->VM_Z*	*VMx_IP*	*custom*	*Allow*	*PCx communication with VMx*
FW2	*VPNcl_IP*	*1025-65535*	*Coffee_Shop->Internet*	*VPNsrv_IP*	*custom*	*Allow*	*Allow VPN tunneling*
FW1	*VPNcl_IP*	*1025-65535*	*Internet->VM_Z*	*VPNsrv_IP*	*custom*	*Allow*	*Allow VPN tunneling*

2. [2 points] Checking the logs of the company, especially those of the NFC reader, it appears that some NFC cards spent more money than their balance (i.e., the sum of the sales to some NFC IDs is higher than the sum of their top ups). You are aware that the encryption on the NFC card uses a 32 bits shared key between all the cards and all the vending machines. It is evident that there is a security issue, can you briefly explain how it has been abused? Propose a better method to handle NFC payment, knowing that an internet connection is already present for the credit card payment.

Avoid storing the actual balance on the card, but use the card only as an ID of the user, and check in the backend whether that user has a credit.

3. [2 points] The network logs of one of the coffee shops are as follows. Can you explain which attack has been put in place and how to mitigate it?

Timestamp	Src IP:port	Dst IP:port	Description
+00,000s	*VM1_IP:80*	*AS_IP:80*	*TCP SYN*
+00,001s	*AS_IP:80*	*VM1_IP:80*	*TCP SYN ACK*
+00,002s	*VM1_IP:80*	*AS_IP:80*	*TCP ACK*
+00,003s	*VM1_IP:80*	*AS_IP:80*	*Creditcard Payment request, ID 0xDEADBEEF [+ payment data]*
+00,103s	*AS_IP:80*	*VM1_IP:80*	*Credit Card Payment confirmation, ID 0xDEADBEEF*

+10,010s	*VM2_IP:80*	*AS_IP:80*	*TCP SYN*
+10,011s	*AS_IP:80*	*VM2_IP:80*	*TCP SYN ACK*
+10,012s	*VM2_IP:80*	*AS_IP:80*	*TCP ACK*
+10,013s	*VM2_IP:80*	*AS_IP:80*	*Creditcard Payment request, ID 0xAAAABBBB[+ payment data]*
+10,014s	*AS_IP:80*	*VM2_IP:80*	*Credit Card Payment confirmation, ID 0xAAAABBBB*
+10,111s	*AS_IP:80*	*VM2_IP:80*	*Credit Card Payment rejection, ID 0xAAAABBBB*

Attack is a MITM implemented on the coffee shop premises by spoofing the victim: since the communication is on http and not encrypted, the attacker can see the TCP SYN/ACK numbers and generate the correct one, anticipating the backend regarding the outcome of the payment confirmation.

Exercise 6 - Malware (6 points)

Consider the malware family **V,** which employs the following metamorphic engine:

- **It randomly selects 30%** of the add instructions and substitutes them with sub instructions, flipping the sign of the second operand.
  For example:
  i. add eax, 0x01 -> sub eax, 0xffffffff
  ii. add ebx, 0xffffffff -> sub ebx, 0x01
- **It randomly selects 30%** of the sub instructions and substitutes them with add instructions, flipping the sign of the second operand.
  For example:
  i. sub eax, 0x01 -> add eax, 0xffffffff
  ii. sub ebx, 0xffffffff -> add ebx, 0x01

6.1 [2 points] Is this metamorphic engine sufficient for evading signature-based detection? Motivate your answer.

No, only 30 percent of the add/sub instructions are changed. Several sections of code are always left unmodified and can, therefore, be used for extracting signatures.

6.2 [2 points] The authors have also released a second version of the same malware family, named **VV**, where the metamorphic engine changes **all** the add/sub instructions according to the scheme described above. Do you think this metamorphic engine is better at evading signature-based detection? Motivate your answer.

No

This obfuscation engine can generate only one variation of the original code.
Executing the malware sample a second time will generate the original version.

6.3 [2 points] Excluding byte-level signature-based detection, how would you statically detect malware samples belonging to the **VV** family?
(N.B., by byte-level it is meant that signatures are extracted from the file's bytes. It is essentially the classic signature-based detection scheme).

Given that VV's engine can only generate one variation of the original sample, an AV could simply compute the hash of the file and check if it equals one of the two versions' hashes.